НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Інститут прикладного системного аналізу
Кафедра системного проектування

Звіт

про виконання комп'ютерного практикуму №8
з дисципліни «Основи молекулярної біології та біоінформатики»

студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович
Варіант 24

Київ-2024

Модифікований алгоритм локального вирівнювання

	Послідовність x	*Послідовність y*
24	VADTPVT	VAGPVAV

Завдання до КП.

1)За допомогою модифікованого с визначити фрагмент (паттерн) послідовності x(див файл SEQ1.docx), що зустрічається один або декілька разів у послідовності y. Для пошуку локальних збігів (повторів) амінокислотні залишки послідовності x повинні формувати рядки, а послідовності y - стовпчики матриці динамічного програмування. Для отримання оптимального вирівнюванні необхідно використовувати матрицю замін амінокислот Blosum 62. Порогове значення T = 20. Як зміниться результат вирівнювання, якщо збільшити або зменшити порогове значення T ? При якому значенні T буде знайдена максимальна кількість збігів фрагменту послідовності x у послідовності y ? При виконанні роботи необхідно спочатку заповнити: 1. Матрицю динамічного програмування з урахуванням порогового значення T=20;

2)Змінити поріг T до такого значення, при якому будуть ідентифіковані повтори фрагмента послідовності x у послідовності y. В матрицях обов`язково позначити напрям отримання максимальної ваги. Кінцеве вирівнювання повинно відповідати графу, що був отриманий при проведенні процедури зворотнього проходу.

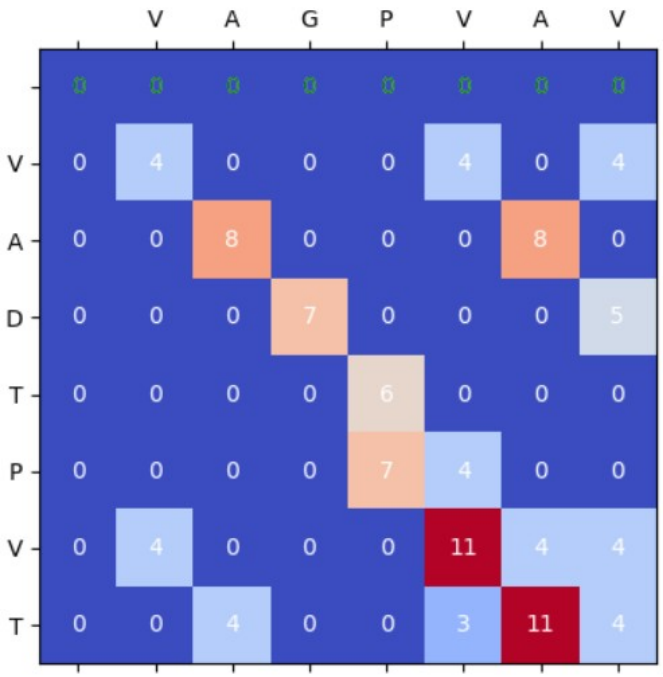
Виконання завдання

Напрям отримання ваги (оцінки) наведену у результаті роботи коду (командній строчці)

T - це порогове значення для пеналізації, яка використовується для вирівнювання послідовностей. В контексті динамічного програмування для локального вирівнювання послідовностей, порогове значення T використовується для зменшення значень у матриці F, щоб врахувати різні впливи пеналізації на вирівнювання. Чим більше значення T, тим сильніше впливає пеналізація на вирівнювання, що може зменшувати ймовірність знаходження оптимальних вирівнювань.

Зміна порогового значення T впливає на пеналізацію за невирівняні позиції в послідовностях, що змінює загальну оцінку вирівнювання. При збільшенні значення T збільшуєтья пеналізація, що робить невирівняні позиції більш дорогими -> менше вирівнювань будуть визнані оптимальними, оскількки вирівнювання будуть більш суворо пеналізуватися за розриви. Результат може мати менше збігів або взагалі не мати. При зменшенні значення, зменшується пеналізація за розриви, що дозволяє більшій кількості вирівнювань бути визнаними оптимальними.

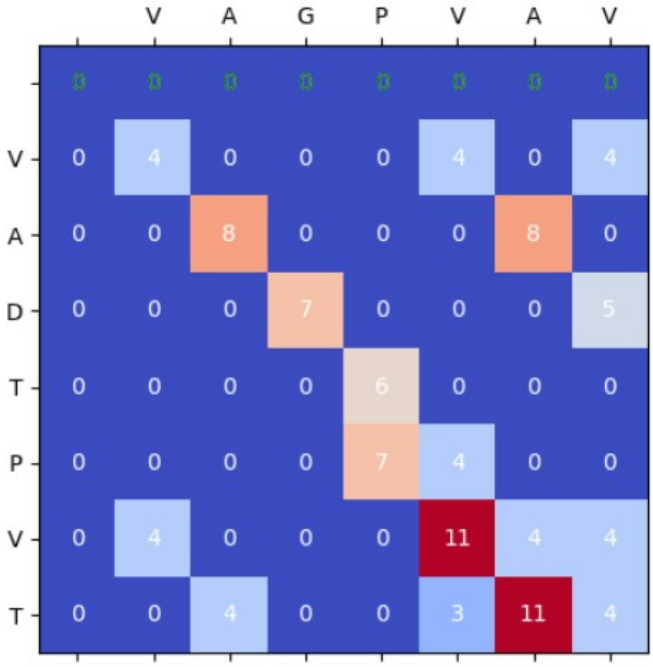
Для T = 20 маємо :




```
Введіть значення T: 20

Модифіковане локальне вирівнювання:
********
VAGPVAV*
Оцінка: 0
Шлях: 0->0->0->0->0->0->0->0
```

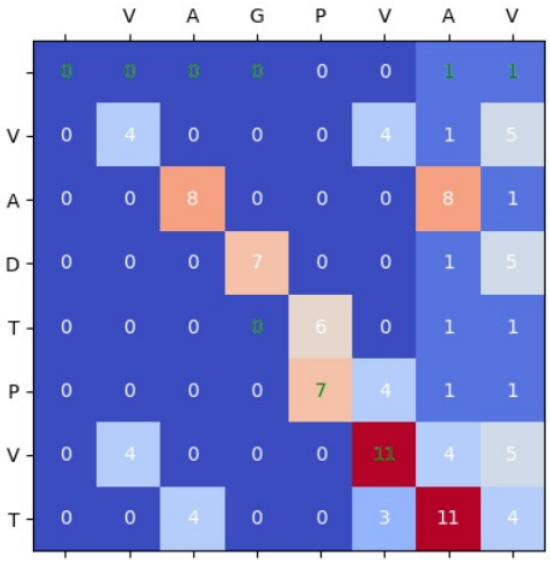
Для T = 15 маємо:



```
Введіть значення T: 15

Модифіковане локальне вирівнювання:
********
VAGPVAV*
Оцінка: 0
Шлях: 0->0->0->0->0->0->0->0
```

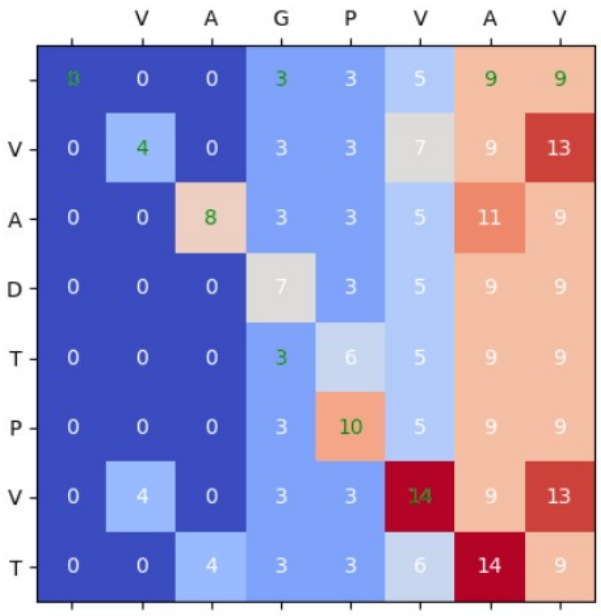
T = 10:




```
Введіть значення T: 10

Модифіковане локальне вирівнювання
****PV***
VAGPPVAV*
Оцінка: 1
Шлях: 1->1->11->7->0->0->0->0->0
```

T = 5:



```
Введіть значення T: 5

Модифіковане локальне вирівнювання:
VA**PV***
VAGPPVAV*
Оцінка: 9
Шлях: 9->9->14->10->3->3->8->4->0
```

З наведених вище результатів роботи коду, можна зробити висновок, що велике значення T – гірше знаходження збігів

Висновок.

У ході виконання лабораторної роботи №8, було написано код модифікованого алгоритму вирівнювання (локальне) на мові Python. Для x= VADTPVT та y =VAGPVAV були використані різні значення (порогові значення для пеналізації) T . Було визначено що при T = 5 ми знаходили максимальну кількість збігів.

Додаток 1

```python
from Bio.Align import substitution_matrices
import numpy as np
import matplotlib.pyplot as plt

blosum50 = substitution_matrices.load("BLOSUM62")


T = int(input("Введіть значення T: "))

x = "VADTPVT"
y = "VAGPVAV"
d = 8

n, m = len(x), len(y)
F = np.zeros((n + 1, m + 2), dtype=int)
Trace = [[0 for _ in range(m + 2)] for _ in range(n + 1)]

for j in range(1, m + 1):
    i0 = 0
    max_score = F[0][j - 1]
    for i in range(0, n + 1):
        current_score = F[i][j - 1] - T
        if current_score > max_score:
            max_score = current_score
            i0 = i
    F[0][j] = max_score
    t = (i0, j - 1)
    Trace[0][j] = t

    for i in range(1, n + 1):
        top = F[0][j]
        insert = F[i - 1][j - 1] + blosum50[x[i - 1], y[j - 1]]
        insert_x = F[i - 1][j] - d
        insert_y = F[i][j - 1] - d
```

```
            max_score = max(top, insert, insert_x, insert_y)
            if max_score == top:
                Trace[i][j] = [0, j]
            elif max_score == insert:
                Trace[i][j] = [i - 1, j - 1]
            elif max_score == insert_x:
                Trace[i][j] = [i - 1, j]
            elif max_score == insert_y:
                Trace[i][j] = [i, j - 1]
            F[i][j] = max_score

    i0 = 0
    j = m + 1
    max_score = F[0][j - 1]
    for i in range(0, n + 1):
        current_score = F[i][j - 1] - T
        if current_score > max_score:
            max_score = current_score
            i0 = i

    F[0][m + 1] = max_score
    Trace[0][m + 1] = (i0, m)

    start_pos = (0, m + 1)
    aligned_x = ""
    aligned_y = ""
    i, j = start_pos
    path = []

    while Trace[i][j] != 0:
        path.append(F[i][j])
        if Trace[i][j] == [i - 1, j - 1]:
            aligned_x = x[i - 1] + aligned_x
            aligned_y = y[j - 1] + aligned_y
            i, j = Trace[i][j]
        elif Trace[i][j] == [i - 1, j]:
            i, j = Trace[i][j]
            if i > 0:
                aligned_x = x[i - 1] + aligned_x
                aligned_y = '-' + aligned_y
        elif Trace[i][j] == [i, j - 1]:
            i, j = Trace[i][j]
            if j > 0:
                aligned_x = '-' + aligned_x
                aligned_y = y[j - 1] + aligned_y
        else:
            i, j = Trace[i][j]
            aligned_x = '*' + aligned_x
            if j < len(y):
                aligned_y = y[j] + aligned_y
            else:
                aligned_y = '*' + aligned_y

    path.append(F[i][j])
    path.pop(0)
    path.reverse()

    print("\nМодифіковане локальне вирівнювання:")
    print(aligned_x)
    print(aligned_y)
    print("Оцінка:", F[0][m + 1])
    print("Шлях:", "->".join(map(str, path[::-1])))

    n, m = len(x), len(y)
    fig, ax = plt.subplots()
```

```
cax = ax.matshow(F[:, :m + 1], cmap="coolwarm")
ax.set_xticks(range(m + 1))
ax.set_yticks(range(n + 1))
ax.set_xticklabels([''] + list(y))
ax.set_yticklabels([''] + list(x))

for i in range(n + 1):
    for j in range(m + 1):
        ax.text(j, i, F[i, j], va='center', ha='center', color='white')

i, j = start_pos
while Trace[i][j] != 0:
    if j < m + 1:
        ax.text(j, i, F[i][j], va='center', ha='center', color='green')
    i, j = Trace[i][j]
if j < m + 1:
    ax.text(j, i, F[i][j], va='center', ha='center', color='green')

plt.show()
```